LAPORAN ANALISIS METADATA DAN DATA GAMBAR MENGGUNAKAN PYTHON

Nama : Shela Rahma Fitri
NIM : 24260012
Semester : 5
Mata Kuliah : Pengolahan Citra Digital

BAB I PENDAHULUAN

Foto digital tampak sebagai gambar utuh ketika dilihat di layar, tetapi di dalam komputer foto disimpan sebagai kumpulan titik kecil yang disebut pixel dan tersusun dalam baris dan kolom. Setiap pixel menyimpan nilai angka yang menentukan warnanya. Pada citra berwarna dengan mode RGB, satu pixel diwakili oleh tiga nilai, yaitu komponen Red (R), Green (G), dan Blue (B), yang masing-masing berada pada rentang 0 sampai 255 untuk data 8 bit per kanal.

Selain data pixel, file foto dari kamera ponsel umumnya menyimpan metadata EXIF (Exchangeable Image File Format), misalnya merek dan model perangkat, waktu pencatatan, dan, apabila tersedia, koordinat GPS. Metadata tidak memengaruhi tampilan foto, tetapi tersimpan dalam file yang sama dan dapat memuat informasi yang bersifat pribadi, seperti waktu dan lokasi. Karena itu, isi metadata perlu dipahami bersama dengan data pixel.

Projek ini menganalisis satu file foto bernama foto.jpg menggunakan Python pada Jupyter Notebook dengan library Pillow, NumPy, Matplotlib, Pandas, serta modul os dan IPython.display.Seluruh kode, output, dan angka pada laporan ini diambil dari notebook tersebut. Analisis dilakukan bertahap: membaca dan menampilkan foto, memeriksa informasi dasar, membaca metadata EXIF dan GPS, mengubah foto menjadi array, mengamati nilai pixel, lalu membentuk kembali gambar dari array dan memeriksa kesamaannya dengan gambar asli.

1.2 Rumusan Masalah

Fokus analisis pada projek ini dirumuskan dalam pertanyaan berikut.

1. Informasi dasar apa saja yang dapat dibaca dari file foto, yaitu nama file, format, ukuran file, resolusi, mode warna, dan jumlah pixel?
2. Metadata EXIF apa yang tersimpan pada foto, dan bagaimana membedakan metadata EXIF utama dengan informasi GPS?
3. Bagaimana koordinat GPS dibaca dan dikonversi dari format derajat, menit, dan detik menjadi derajat desimal untuk ditampilkan pada peta?
4. Bagaimana membuktikan bahwa foto tersusun dari pixel bernilai angka melalui array NumPy?
5. Bagaimana hubungan antara koordinat pixel, nilai RGB, dan warna yang ditampilkan?
6. Apakah gambar yang dibentuk kembali dari array memiliki nilai pixel yang sama dengan gambar asli?

BAB II IMPLEMENTASI DAN PEMBAHASAN PROGRAM

Bab ini membahas 13 tahap kode pada notebook. Tahap 1 sampai 3 menyiapkan library, membaca foto, dan memeriksa informasi dasarnya. Tahap 4 sampai 6 membahas metadata EXIF dan GPS. Tahap 7 sampai 12 membuktikan bahwa foto tersusun dari pixel bernilai angka, dan Tahap 13 memeriksa rekonstruksi gambar dari array.

2.1 Tahap 1 — Persiapan Library

1. Tujuan Kode

Menyiapkan library yang dibutuhkan pada tahap-tahap berikutnya, yaitu untuk membaca gambar dan metadata, menampilkan tabel dan tautan, mengolah array angka, menampilkan gambar, serta mengakses informasi file.

2. Screenshot Kode Program

```
from PIL import Image, ExifTags
from IPython.display import display, HTML
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os
```

Gambar 1. Kode program tahap 1 (persiapan library)

3. Penjelasan Setiap Baris Kode

Baris 1

```
from PIL import Image, ExifTags
```

Mengimpor dua komponen dari library Pillow (PIL). `Image` dipakai untuk membuka file gambar dan membentuk objek gambar (`Image.open()` dan `Image.fromarray()`). `ExifTags` menyediakan kamus `TAGS` dan `GPSTAGS` yang memetakan ID tag EXIF berupa angka ke nama yang mudah dibaca, serta konstanta `ExifTags.IFD.GPSInfo` yang dipakai untuk mengakses blok metadata GPS.

Baris 2

```
from IPython.display import display, HTML
```

Mengimpor `display` untuk menampilkan objek seperti tabel Pandas dan peta pada output sel, serta `HTML` untuk menampilkan teks HTML, yang dalam notebook ini dipakai untuk membuat tautan Google Maps.

Baris 3

```
import numpy as np
```

Mengimpor NumPy dengan alias `np`. NumPy dipakai untuk mengubah gambar menjadi array dan membandingkan dua array.

Baris 4

```
import matplotlib.pyplot as plt
```

Mengimpor modul `pyplot` dari Matplotlib dengan alias `plt` untuk menampilkan gambar dan visualisasi.

Baris 5

```
import pandas as pd
```

Mengimpor Pandas dengan alias `pd` untuk menyusun hasil analisis dalam bentuk tabel (DataFrame).

Baris 6

```
import os
```

Mengimpor modul `os` bawaan Python. Pada notebook ini `os.path.basename()` dan `os.path.getsize()` dipakai untuk mengambil nama file dan ukuran file.

4. Hasil Program

Sel ini tidak menampilkan output. Pengaruhnya adalah library tersedia untuk dipakai pada sel-sel berikutnya.

5. Analisis Hasil

Pembagian tugas library sudah jelas, sebagaimana diringkas pada Tabel 1. Tanpa tahap ini, fungsi seperti `Image.open()`, `np.array()`, `plt.imshow()`, dan `pd.DataFrame()` pada tahap berikutnya tidak dapat dipanggil.

Tabel 1. Library yang digunakan dan fungsinya

Library	Perintah impor (baris)	Fungsi pada projek
Pillow (PIL)	`from PIL import Image, ExifTags` (1)	Membuka gambar, membaca metadata EXIF, dan membentuk gambar dari array
IPython.display	`from IPython.display import display, HTML` (2)	Menampilkan tabel dan tautan HTML pada output sel
NumPy	`import numpy as np` (3)	Mengubah gambar menjadi array dan membandingkan array
Matplotlib	`import matplotlib.pyplot as plt` (4)	Menampilkan gambar dan visualisasi

Library	Perintah impor (baris)	Fungsi pada projek
Pandas	`import pandas as pd` (5)	Menyusun hasil analisis dalam bentuk tabel
os	`import os` (6)	Mengambil nama dan ukuran file
folium	`import folium` (Tahap 6, dalam blok try)	Menampilkan peta interaktif (opsional; belum terpasang pada output notebook)

Dua library yang bukan pengolah gambar perlu dijelaskan secara singkat. `os` adalah modul bawaan Python untuk berinteraksi dengan sistem berkas dan jalur file; pada notebook ini dipakai pada Tahap 3 melalui `os.path.basename()` dan `os.path.getsize()`. `IPython.display` adalah bagian dari lingkungan IPython/Jupyter: fungsi `display()` dipakai pada Tahap 4, 6, dan 11 untuk menampilkan tabel serta peta, sedangkan `HTML` dipakai pada Tahap 6 untuk menampilkan tautan Google Maps. Selain library pada sel persiapan, `folium` diimpor di dalam blok `try` pada Tahap 6 dan bersifat opsional; `import pandas as pd` juga ditulis ulang pada Tahap 11 tanpa mengubah hasil.

Catatan. Penjelasan library pada bagian Markdown notebook mencantumkan Pillow, NumPy, Matplotlib, Pandas, dan os, tetapi tidak mencantumkan `IPython.display`. Library tersebut tetap digunakan pada kode (baris 2), sehingga dimasukkan pada Tabel 1.

2.2 Tahap 2 — Membaca dan Menampilkan Citra

1. Tujuan Kode

Membuka file foto asli, menyimpannya dalam variabel `image`, dan menampilkannya agar objek yang dianalisis dapat dilihat secara visual.

2. Screenshot Kode Program

```
# Membuka file citra asli
image = Image.open("foto/foto.jpg")

# Menampilkan citra dengan ukuran tampilan lebih kecil
plt.figure(figsize=(4, 5))
plt.imshow(image)
plt.axis("off")
plt.show()
```

Gambar 2. Kode program tahap 2 (membaca dan menampilkan citra)

3. Penjelasan Setiap Baris Kode

Baris 1

```
# Membuka file citra asli
image = Image.open("foto/foto.jpg")
```

Baris 1 adalah , `Image.open()` membuka file gambar dengan lokasi `"foto/foto.jpg"`, yaitu file `foto.jpg` di dalam folder `foto` relatif terhadap lokasi notebook. Hasilnya disimpan pada variabel `image`, sebuah objek gambar Pillow yang dipakai pada hampir seluruh tahap berikutnya.

Baris 2

```
# Menampilkan citra dengan ukuran tampilan lebih kecil
plt.figure(figsize=(4, 5))
```

`plt.figure(figsize=(4, 5))` membuat kanvas tampilan berukuran 4 × 5 inci. Pengaturan ini hanya memengaruhi ukuran tampilan di notebook dan tidak mengubah ukuran, resolusi, maupun nilai pixel pada file foto asli.

Baris 3

```
plt.imshow(image)
```

Menampilkan objek `image` pada area plot Matplotlib.

Baris 4

```
plt.axis("off")
```

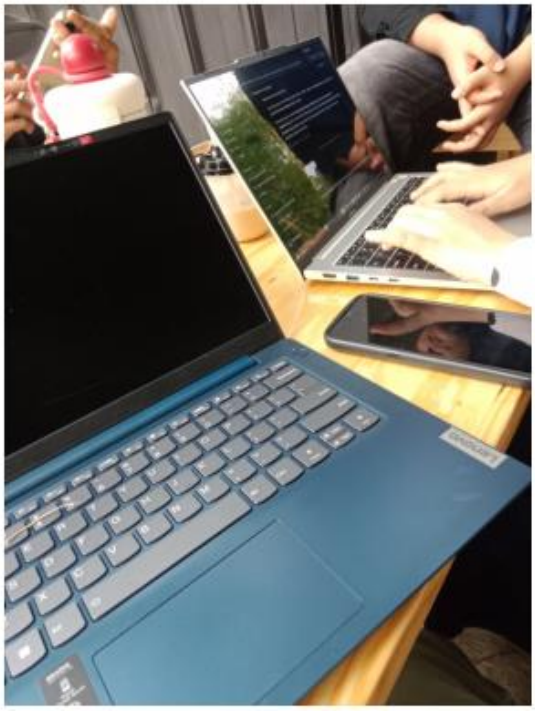
Menyembunyikan sumbu koordinat agar tampilan foto lebih bersih.

Baris 5

```
plt.show()
```

Menampilkan hasil visualisasi pada notebook.

4. Hasil Program



Gambar 3. Hasil tampilan foto pada tahap 2

5. Analisis Hasil

Foto tampil dalam orientasi potret (tinggi lebih besar daripada lebar). Isi foto memperlihatkan laptop, ponsel, dan tangan beberapa orang di atas meja kayu. Tidak muncul pesan kesalahan, sehingga file ditemukan pada lokasi yang ditulis dan formatnya dapat dibaca Pillow. Tampilan ini masih berupa gambar yang dilihat mata. Bukti bahwa gambar tersusun dari pixel bernilai angka baru dimulai pada Tahap 7. Perlu dicatat bahwa gambar pada Gambar 3 adalah tampilan notebook yang diperkecil, bukan file asli yang berukuran 3120 × 4160 pixel.

2.3 Tahap 3 — Informasi Dasar Citra

1. Tujuan Kode

Mengetahui informasi dasar file foto, yaitu nama file, format, ukuran file, resolusi, mode warna, dan jumlah keseluruhan pixel, sebagai dasar sebelum analisis metadata dan data numerik.

2. Screenshot Kode Program

```python
# Lokasi file citra
file_path = "foto/foto.jpg"

# Informasi dasar citra
nama_file = os.path.basename(file_path)
format_file = image.format
ukuran_file = os.path.getsize(file_path)
lebar, tinggi = image.size
mode_warna = image.mode
jumlah_pixel = lebar * tinggi

print("Nama file       :", nama_file)
print("Format          :", format_file)
print("Ukuran file     :", ukuran_file, "bytes")
print("Resolusi        :", lebar, "x", tinggi, "pixel")
print("Mode warna      :", mode_warna)
print("Jumlah piksel   :", jumlah_pixel, "pixel")
```

Gambar 4. Kode program tahap 3 (informasi dasar citra)

3. Penjelasan Setiap Baris Kode

Baris 1

```
# Lokasi file citra
file_path = "foto/foto.jpg"
```

Variabel `file_path` menyimpan lokasi file foto sebagai teks sehingga dapat dipakai ulang pada beberapa perintah.

Baris 2

```
# Informasi dasar citra
nama_file = os.path.basename(file_path)
```

`os.path.basename()` mengambil nama file dari lokasi lengkapnya, sehingga `nama_file` bernilai `foto.jpg` tanpa nama folder.

Baris 3

```
format_file = image.format
```

Atribut `image.format` memberi format file asli yang dikenali Pillow.

Baris 4

```
ukuran_file = os.path.getsize(file_path)
```

`os.path.getsize()` mengembalikan ukuran file pada penyimpanan dalam satuan byte.

Baris 5

```
lebar, tinggi = image.size
```

`image.size` berisi pasangan (lebar, tinggi) dalam pixel. Kedua nilainya langsung dipisahkan ke variabel `lebar` dan `tinggi` (unpacking).

Baris 6

```
mode_warna = image.mode
```

`image.mode` memberi mode warna, yaitu cara informasi warna disimpan (misalnya RGB).

Baris 7

```
jumlah_pixel = lebar * tinggi
```

Jumlah pixel dihitung dengan mengalikan lebar dan tinggi.

Baris 8-13

```
print("Nama file       :", nama_file)
print("Format          :", format_file)
print("Ukuran file     :", ukuran_file, "bytes")
print("Resolusi        :", lebar, "x", tinggi, "pixel")
print("Mode warna      :", mode_warna)
print("Jumlah piksel   :", jumlah_pixel, "pixel")
```

Enam perintah `print()` menampilkan hasil dengan label. Spasi pada label hanya dipakai agar titik dua sejajar.

4. Hasil Program

```
Nama file       : foto.jpg
Format          : JPEG
Ukuran file     : 3452320 bytes
Resolusi        : 3120 x 4160 pixel
Mode warna      : RGB
Jumlah piksel   : 12979200 pixel
```

Gambar 5. Hasil program tahap 3 (informasi dasar citra)

5. Analisis Hasil

File yang dianalisis bernama `foto.jpg` berformat JPEG dengan ukuran file 3.452.320 byte atau sekitar 3,45 MB (dengan 1 MB = 1.000.000 byte). Resolusi foto adalah 3120 × 4160 pixel (lebar × tinggi), sehingga jumlah pixelnya:

$$3120 \times 4160 = 12.979.200 \text{ pixel}$$

atau sekitar 13 juta pixel. Mode warna RGB berarti setiap pixel memiliki tiga komponen, yaitu Red, Green, dan Blue. Istilah "resolusi" pada tahap ini berarti jumlah pixel pada lebar dan tinggi gambar. Pengertian ini berbeda dari nilai `XResolution` dan `YResolution` pada metadata EXIF yang dibahas pada Tahap 4.

Sebagai perhitungan penjelas, apabila setiap pixel disimpan tanpa kompresi dengan 3 byte (satu byte per kanal), data gambar membutuhkan 12.979.200 × 3 = 38.937.600 byte. Ukuran file JPEG yang jauh lebih kecil menunjukkan bahwa file tersebut telah dikompresi. Perhitungan ini bukan bagian dari kode notebook.

2.4 Tahap 4 — Analisis Metadata EXIF

1. Tujuan Kode

Membaca metadata EXIF yang tersimpan pada file foto, menghitung jumlah entrinya, dan menyusunnya dalam tabel dengan nama tag yang mudah dibaca.

2. Screenshot Kode Program

```
# Membaca metadata EXIF
exif_data = image.getexif()

# Menampilkan jumlah metadata
print("Jumlah data EXIF:", len(exif_data))

# Menyusun metadata dalam bentuk tabel
data_tabel = []

for tag_id, value in exif_data.items():
    tag_name = ExifTags.TAGS.get(tag_id, tag_id)
    data_tabel.append([tag_name, value])

# Menampilkan tabel metadata
display(pd.DataFrame(data_tabel, columns=["Metadata", "Nilai"]))
```

Gambar 6. Kode program tahap 4 (metadata EXIF)

3. Penjelasan Setiap Baris Kode

Baris 1

```
# Membaca metadata EXIF
exif_data = image.getexif()
```

`getexif()` adalah metode Pillow untuk mengambil metadata EXIF, yaitu data tambahan yang disisipkan perangkat ke dalam file foto. Hasilnya disimpan pada `exif_data` dengan struktur mirip kamus (dictionary): kunci berupa ID tag angka dan nilai berupa isi metadata.

Baris 2

```
# Menampilkan jumlah metadata
print("Jumlah data EXIF:", len(exif_data))
```

`len(exif_data)` menghitung banyaknya entri metadata, lalu `print()` menampilkannya.

Baris 3

```
# Menyusun metadata dalam bentuk tabel
data_tabel = []
```

Membuat daftar kosong `data_tabel` sebagai wadah pasangan nama metadata dan nilainya sebelum diubah menjadi tabel.

Baris 4

```
for tag_id, value in exif_data.items():
```

Perulangan yang membaca seluruh isi `exif_data`. Metode `items()` memberikan pasangan (ID tag, nilai), yang ditampung bergantian pada `tag_id` dan `value`.

Baris 5

```
    tag_name = ExifTags.TAGS.get(tag_id, tag_id)
```

`ExifTags.TAGS` adalah kamus yang memetakan ID angka ke nama tag. Metode `get(tag_id, tag_id)` mengambil nama tag, dan jika ID tidak ditemukan, yang dipakai adalah ID itu sendiri sehingga program tidak berhenti dengan kesalahan.

Baris 6

```
    data_tabel.append([tag_name, value])
```

Menambahkan satu baris berisi nama metadata dan nilainya ke `data_tabel`. Setelah perulangan selesai, daftar ini memuat seluruh entri metadata.

Baris 7

```
# Menampilkan tabel metadata
display(pd.DataFrame(data_tabel, columns=["Metadata", "Nilai"]))
```

`pd.DataFrame()` mengubah `data_tabel` menjadi tabel dengan dua kolom bernama `Metadata` dan `Nilai`, dan `display()` menampilkannya pada output sel.

4. Hasil Program

```
Jumlah data EXIF: 14
```

Gambar 7. Jumlah data EXIF pada tahap 4

	Metadata	Nilai
0	ImageWidth	3120
1	ImageLength	4160
2	GPSInfo	1092
3	ResolutionUnit	2
4	ExifOffset	346
5	ImageDescription	
6	Make	vivo
7	Model	V2204
8	Software	MediaTek Camera Application
9	Orientation	0
10	DateTime	2026:10:06 16:18:26
11	YCbCrPositioning	2
12	XResolution	72.0
13	YResolution	72.0

Gambar 8. Tabel metadata EXIF pada tahap 4

Pada tampilan gambar ini, karakter null (`\x00`) yang menyertai nilai teks pada beberapa baris tidak ditampilkan karena tidak terlihat.

5. Analisis Hasil

Foto memiliki 14 entri metadata EXIF utama. Makna entri tersebut diringkas pada Tabel 2. Nilai `ImageWidth` 3120 dan `ImageLength` 4160 sama dengan ukuran yang dibaca dari `image.size` pada Tahap 3.

Tabel 2. Metadata EXIF utama dan maknanya

Metadata	Nilai	Makna dan catatan interpretasi
ImageWidth	3120	Lebar gambar dalam pixel.
ImageLength	4160	Tinggi gambar dalam pixel.
GPSInfo	1092	Penunjuk (offset) ke blok metadata GPS, bukan koordinat. Isi GPS dibaca pada Tahap 5.
ResolutionUnit	2	Satuan untuk XResolution dan YResolution; nilai 2 berarti inci.
ExifOffset	346	Penunjuk ke blok EXIF lanjutan (sub-IFD) yang tidak dibaca pada notebook ini.
ImageDescription	(kosong)	Deskripsi gambar tidak diisi; nilainya hanya karakter null.

Metadata	Nilai	Makna dan catatan interpretasi
Make	vivo	Merek perangkat yang tercatat pada file.
Model	V2204	Model perangkat yang tercatat pada file.
Software	MediaTek Camera Application	Nama perangkat lunak yang tercatat pada file.
Orientation	0	Nilai standar orientasi adalah 1 sampai 8, sehingga nilai 0 tidak dapat ditafsirkan sebagai arah rotasi tertentu.
DateTime	2026:10:06 16:18:26	Tanggal dan waktu (TTTT:BB:HH JJ:MM:DD) yang dicatat perangkat; zona waktu tidak tercatat pada tag ini.
YCbCrPositioning	2	Posisi sampel warna (chroma) pada pengkodean JPEG; nilai 2 berarti co-sited.
XResolution	72.0	72 pixel per inci pada arah horizontal (ResolutionUnit = 2).
YResolution	72.0	72 pixel per inci pada arah vertikal.

Beberapa hal perlu diperhatikan dalam menafsirkan tabel tersebut.

1. **Angka pada GPSInfo dan ExifOffset bukan data isi.** Keduanya berupa penunjuk letak (offset) ke blok metadata lain di dalam file. Karena itu angka 1092 pada `GPSInfo` bukan lintang atau bujur, dan koordinat baru terbaca setelah blok tersebut dibuka pada Tahap 5.
2. **Kode hanya membaca entri utama.** Entri pada blok yang ditunjuk `ExifOffset` (misalnya data pengambilan gambar seperti ISO atau bukaan, jika ada) tidak dibaca oleh kode ini. Dengan demikian, tidak dapat disimpulkan apakah informasi tersebut tersedia pada foto.
3. **Make, Model, dan Software.** Nilai ini menunjukkan perangkat dan aplikasi yang tercatat pada file, yaitu vivo, V2204, dan MediaTek Camera Application. Metadata dapat diubah oleh aplikasi lain, sehingga nilai ini dipahami sebagai catatan pada file, bukan bukti mutlak perangkat pengambil foto.
4. **Orientation bernilai 0.** Nilai ini di luar nilai standar, sehingga tidak dipakai untuk menafsirkan rotasi. Pada pembacaan Pillow, ukuran gambar dan tampilan foto tetap berorientasi potret.
5. **XResolution dan YResolution** bernilai 72.0 menunjukkan kerapatan nominal 72 pixel per inci untuk keperluan tampil atau cetak. Nilai ini tidak sama dengan dimensi 3120 × 4160 pixel dan tidak menunjukkan kualitas gambar.

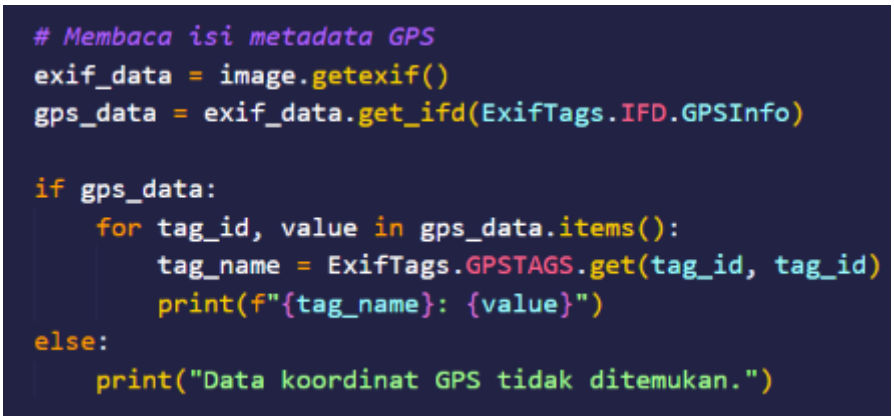
6. **Karakter null.** Nilai teks seperti `Make`, `Model`, dan `Software` pada output mentah diikuti karakter null yang tidak terlihat. Karakter tersebut hanya pengisi atau penutup dan bukan bagian dari isi informasi.

2.5 Tahap 5 — Membaca Metadata GPS

1. Tujuan Kode

Membuka blok metadata GPS yang ditunjuk oleh tag `GPSInfo` pada Tahap 4, kemudian menampilkan seluruh isinya dengan nama tag GPS yang mudah dibaca.

2. Screenshot Kode Program



Gambar 9. Kode program tahap 5 (membaca metadata GPS)

3. Penjelasan Setiap Baris Kode

Baris 1-2

```
# Membaca isi metadata GPS
exif_data = image.getexif()
gps_data = exif_data.get_ifd(ExifTags.IFD.GPSInfo)
```

Baris 2 membaca ulang metadata EXIF utama. Baris 3 memanggil `get_ifd()` dengan konstanta `ExifTags.IFD.GPSInfo` untuk membuka blok GPS yang ditunjuk oleh tag `GPSInfo`. Hasilnya, `gps_data`, adalah kamus berisi tag GPS. Inilah perbedaan metadata EXIF utama (Tahap 4, `GPSInfo` hanya berisi angka penunjuk) dengan blok GPS (tahap ini, berisi koordinat dan keterangan lain).

Baris 3-6

```
if gps_data:
    for tag_id, value in gps_data.items():
        tag_name = ExifTags.GPSTAGS.get(tag_id, tag_id)
        print(f"{tag_name}: {value}")
```

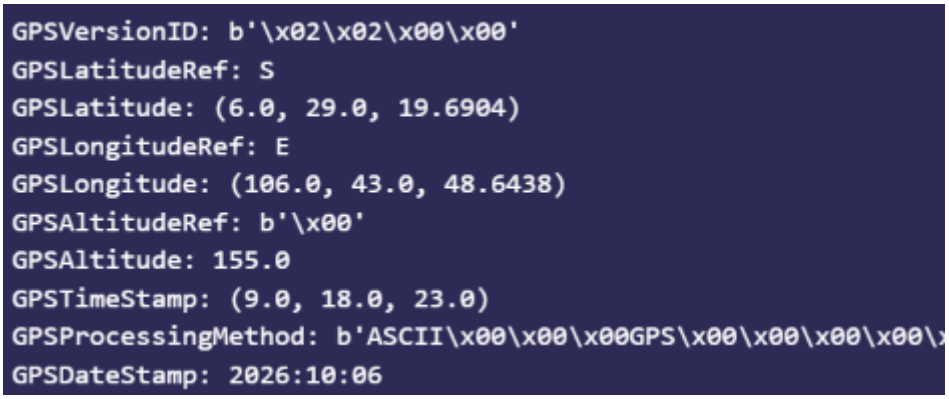
Kondisi `if gps_data:` bernilai benar jika blok GPS tidak kosong. Perulangan membaca setiap pasangan (ID tag, nilai). `ExifTags.GPSTAGS` adalah kamus nama tag khusus GPS, dan `get(tag_id, tag_id)` memakai ID itu sendiri bila nama tidak ditemukan. `print(f"{tag_name}: {value}")` mencetak nama tag dan nilainya dengan f-string.

Baris 7

```
else:
    print("Data koordinat GPS tidak ditemukan.")
```

Cabang `else` hanya berjalan bila blok GPS kosong, dengan mencetak pesan bahwa data GPS tidak ditemukan. Cabang ini tidak berjalan pada foto ini karena data GPS ditemukan.

4. Hasil Program



Gambar 10. Hasil program tahap 5 (isi metadata GPS)

Baris `GPSProcessingMethod` yang panjang dipecah ke beberapa baris pada gambar agar terbaca; isinya sama dengan output notebook.

5. Analisis Hasil

Data GPS ditemukan pada foto. Isi dan makna setiap tag diringkas pada Tabel 3.

Tabel 3. Isi metadata GPS dan maknanya

Tag GPS	Nilai terbaca	Makna
GPSVersionID	b'\x02\x02\x00\x00'	Empat byte versi format GPS, yaitu 2, 2, 0, 0 (versi 2.2.0.0).
GPSLatitudeRef	S	Lintang Selatan; koordinat berada di selatan khatulistiwa.
GPSLatitude	(6.0, 29.0, 19.6904)	Lintang dalam derajat, menit, dan detik: 6° 29′ 19,6904″.
GPSLongitudeRef	E	Bujur Timur.
GPSLongitude	(106.0, 43.0, 48.6438)	Bujur dalam derajat, menit, dan detik: 106° 43′ 48,6438″.

Tag GPS	Nilai terbaca	Makna
GPSAltitudeRef	b'\x00'	Kode 0 berarti ketinggian berada di atas permukaan laut.
GPSAltitude	155.0	Ketinggian 155,0 meter menurut ketentuan satuan pada spesifikasi EXIF.
GPSTimeStamp	(9.0, 18.0, 23.0)	Waktu 09:18:23 dalam UTC.
GPSProcessingMethod	b'ASCII\x00\x00\x00GPS\x00…'	Keterangan metode pemrosesan lokasi; lihat penjelasan di bawah.
GPSDateStamp	2026:10:06	Tanggal 6 Oktober 2026 (format TTTT:BB:HH).

Hal-hal yang perlu diperhatikan:

1. **Lintang dan bujur** tersimpan terpisah sebagai angka dan arah. Angka `6.0, 29.0, 19.6904` belum menunjukkan apakah lokasi berada di utara atau selatan. Arah ditentukan oleh `GPSLatitudeRef`, yaitu `S`, sehingga lintang bernilai negatif ketika dikonversi pada Tahap 6. Pada bujur, arah `E` berarti bernilai positif.
2. **GPSProcessingMethod** merupakan data bytes. Pada format EXIF, nilai jenis ini diawali kode karakter sepanjang 8 byte (`ASCII` dan karakter null), lalu diikuti teks. Teks yang terbaca di sini hanya `GPS`, diikuti karakter null pengisi. Informasi ini tidak cukup untuk menyimpulkan cara perangkat memperoleh koordinat, misalnya dari satelit saja atau dibantu jaringan, sehingga laporan ini tidak menafsirkannya lebih jauh.
3. **Waktu GPS dan DateTime.** `GPSTimeStamp` berada dalam UTC. Jika perangkat menggunakan zona waktu UTC+7, 09:18:23 UTC setara dengan 16:18:23, selisih tiga detik dari `DateTime` 16:18:26 pada Tahap 4, dan tanggalnya sama dengan `GPSDateStamp` (2026:10:06). Kesesuaian ini hanya menunjukkan bahwa kedua catatan waktu konsisten. Kesesuaian ini bukan bukti keaslian metadata, dan zona waktu perangkat tidak tercatat pada metadata.

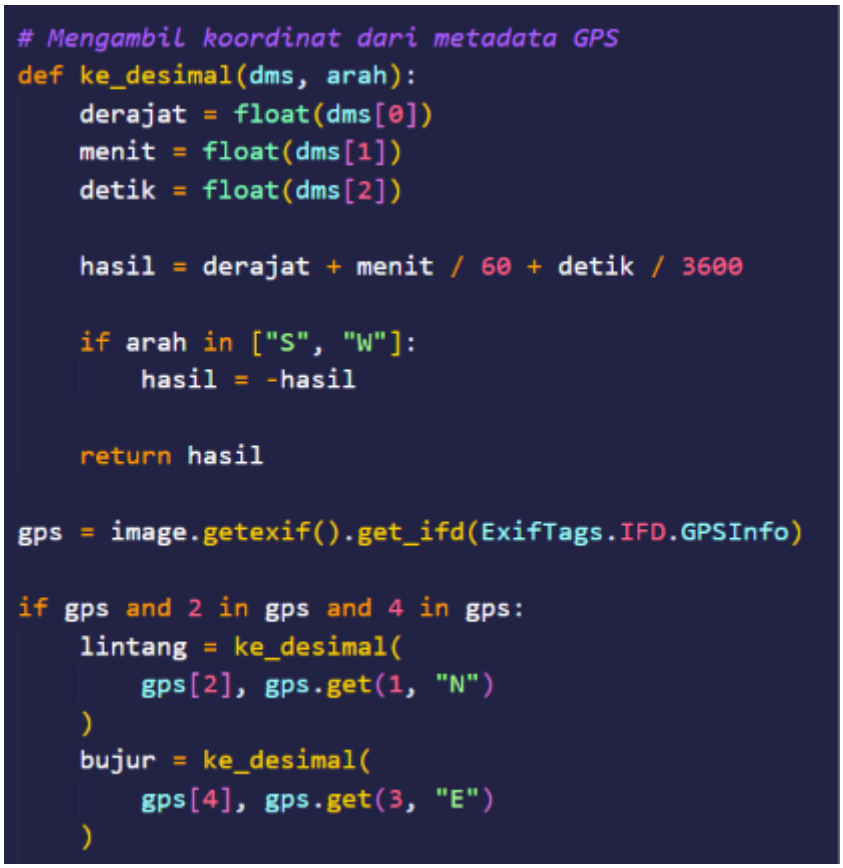
2.6 Tahap 6 — Konversi Koordinat GPS dan Visualisasi Lokasi

1. Tujuan Kode

Mengonversi koordinat GPS dari format derajat, menit, dan detik menjadi derajat desimal, menyusunnya dalam tabel, membuat tautan Google Maps dari koordinat tersebut, dan mencoba menampilkan peta interaktif menggunakan library folium apabila tersedia.

2. Screenshot Kode Program

Kode pada tahap ini cukup panjang sehingga ditampilkan dalam tiga bagian dengan nomor baris yang berlanjut.



```
# Mengambil koordinat dari metadata GPS
def ke_desimal(dms, arah):
    derajat = float(dms[0])
    menit = float(dms[1])
    detik = float(dms[2])

    hasil = derajat + menit / 60 + detik / 3600

    if arah in ["S", "W"]:
        hasil = -hasil

    return hasil

gps = image.getexif().get_ifd(ExifTags.IFD.GPSInfo)

if gps and 2 in gps and 4 in gps:
    lintang = ke_desimal(
        gps[2], gps.get(1, "N")
    )
    bujur = ke_desimal(
        gps[4], gps.get(3, "E")
    )
```

Gambar 11. Kode program tahap 6, bagian 1 (fungsi konversi dan perhitungan koordinat)

```python
# Membuat tautan Google Maps
link_maps = (
    f"https://www.google.com/maps?q={lintang},{bujur}"
)

# Menampilkan tabel koordinat
tabel_gps = pd.DataFrame({
    "Informasi": [
        "Latitude (Lintang)",
        "Longitude (Bujur)",
        "Koordinat Desimal"
    ],
    "Nilai": [
        f"{lintang:.6f}°",
        f"{bujur:.6f}°",
        f"{lintang:.6f}, {bujur:.6f}"
    ]
})

display(tabel_gps)

display(HTML(
    f'<a href="{link_maps}" target="_blank">'
    '📍 Buka lokasi di Google Maps</a>'
))
```

Gambar 12. Kode program tahap 6, bagian 2 (tautan Google Maps dan tabel koordinat)

```
    # Peta interaktif (jika folium terpasang)
    try:
        import folium

        peta = folium.Map(
            location=[lintang, bujur],
            zoom_start=16
        )

        folium.Marker(
            [lintang, bujur],
            tooltip="Lokasi dari metadata foto",
            popup=f"Koordinat: {lintang:.6f}, {bujur:.6f}"
        ).add_to(peta)

        display(peta)

    except ImportError:
        print(
            "Peta interaktif belum tersedia. "
            "Gunakan tautan Google Maps di atas."
        )

else:
    print("Koordinat GPS tidak lengkap atau tidak tersedia.")
```

Gambar 13. Kode program tahap 6, bagian 3 (peta interaktif dan cabang kondisi lain)

3. Penjelasan Setiap Baris Kode

Baris 1–2

```
# Mengambil koordinat dari metadata GPS
def ke_desimal(dms, arah):
```

Mendefinisikan fungsi `ke_desimal` dengan dua parameter: `dms` (tiga nilai derajat, menit, dan detik) dan `arah` (huruf arah mata angin: `N`, `S`, `E`, atau `W`).

Baris 3–5

```
    derajat = float(dms[0])
    menit = float(dms[1])
    detik = float(dms[2])
```

Mengambil elemen pertama, kedua, dan ketiga dari `dms` dan mengubahnya menjadi bilangan desimal (`float`) agar dapat dihitung.

Baris 7

```
    hasil = derajat + menit / 60 + detik / 3600
```

Rumus konversi ke derajat desimal. Satu derajat terdiri atas 60 menit dan satu menit terdiri atas 60 detik, sehingga satu derajat sama dengan 3600 detik.

Baris 9–10

```
    if arah in ["S", "W"]:
        hasil = -hasil
```

Jika arah adalah Selatan (`S`) atau Barat (`W`), hasil dijadikan negatif. Konvensi derajat desimal memberi nilai negatif untuk lintang selatan dan bujur barat.

Baris 12

```
    return hasil
```

Mengembalikan nilai derajat desimal sebagai hasil fungsi.

Baris 14

```
gps = image.getexif().get_ifd(ExifTags.IFD.GPSInfo)
```

Membaca blok GPS seperti pada Tahap 5, tetapi dalam satu baris. Hasilnya disimpan pada `gps`.

Baris 16

```
if gps and 2 in gps and 4 in gps:
```

Memeriksa bahwa blok GPS tidak kosong dan memuat tag nomor 2 (`GPSLatitude`) serta nomor 4 (`GPSLongitude`). Kondisi ini mencegah kesalahan jika koordinat tidak lengkap.

Baris 17–19

```
    lintang = ke_desimal(
        gps[2], gps.get(1, "N")
    )
```

Memanggil `ke_desimal` untuk lintang. `gps[2]` adalah `GPSLatitude`, sedangkan `gps.get(1, "N")` mengambil `GPSLatitudeRef` (tag nomor 1) dengan nilai cadangan `"N"` bila tag tersebut tidak ada.

Baris 20–22

```
    bujur = ke_desimal(
        gps[4], gps.get(3, "E")
    )
```

Cara yang sama untuk bujur: `gps[4]` adalah `GPSLongitude` dan `gps.get(3, "E")` mengambil `GPSLongitudeRef` (tag nomor 3) dengan nilai cadangan `"E"`.

Baris 24–27

```
    # Membuat tautan Google Maps
    link_maps = (
        f"https://www.google.com/maps?q={lintang},{bujur}"
    )
```

Membentuk tautan Google Maps dengan f-string. Parameter `q=` berisi pasangan lintang dan bujur yang disisipkan dari variabel `lintang` dan `bujur`. Tanda kurung pada baris 25–27 hanya membungkus teks.

Baris 29–41

```
    # Menampilkan tabel koordinat
    tabel_gps = pd.DataFrame({
        "Informasi": [
            "Latitude (Lintang)",
            "Longitude (Bujur)",
            "Koordinat Desimal"
        ],
        "Nilai": [
            f"{lintang:.6f}°",
            f"{bujur:.6f}°",
            f"{lintang:.6f}, {bujur:.6f}"
        ]
    })
```

Membuat tabel Pandas dari kamus berisi dua kolom, `Informasi` dan `Nilai`, masing-masing berisi tiga baris. Format `:.6f` menampilkan angka dengan enam angka di belakang koma, dan tanda ° ditambahkan sebagai simbol derajat. Enam desimal hanya mengatur cara penulisan dan tidak menunjukkan ketelitian posisi.

Baris 43

```
    display(tabel_gps)
```

Menampilkan tabel koordinat pada output sel.

Baris 45–48

```
    display(HTML(
        f'<a href="{link_maps}" target="_blank">'
        '📍 Buka lokasi di Google Maps</a>'
    ))
```

Membuat tautan HTML yang menunjuk ke `link_maps`. `target="_blank"` membuat tautan terbuka pada tab baru. Dua literal teks pada baris 46 dan 47 yang berdampingan otomatis digabung oleh Python menjadi satu teks HTML, dan `display()` menampilkannya sebagai tautan yang dapat diklik.

Baris 50–52

```
    # Peta interaktif (jika folium terpasang)
    try:
        import folium
```

Blok `try` mencoba mengimpor library `folium`. Jika library belum terpasang, Python memunculkan `ImportError` yang ditangani pada baris 67.

Baris 54–57

```
        peta = folium.Map(
            location=[lintang, bujur],
            zoom_start=16
        )
```

Membuat objek peta yang berpusat pada koordinat hasil konversi, dengan tingkat zoom 16.

Baris 59–63

```
        folium.Marker(
            [lintang, bujur],
            tooltip="Lokasi dari metadata foto",
            popup=f"Koordinat: {lintang:.6f}, {bujur:.6f}"
        ).add_to(peta)
```

Menambahkan penanda (marker) pada koordinat tersebut. `tooltip` adalah teks yang muncul ketika kursor diarahkan ke penanda, sedangkan `popup` muncul ketika penanda diklik. `add_to(peta)` memasang penanda pada peta.

Baris 65

```
        display(peta)
```

Menampilkan peta interaktif pada output sel.

Baris 67–71

```
    except ImportError:
        print(
            "Peta interaktif belum tersedia. "
            "Gunakan tautan Google Maps di atas."
        )
```

Jika `folium` belum terpasang, program tidak berhenti, tetapi mencetak pesan agar pengguna memakai tautan Google Maps.

Baris 73–74

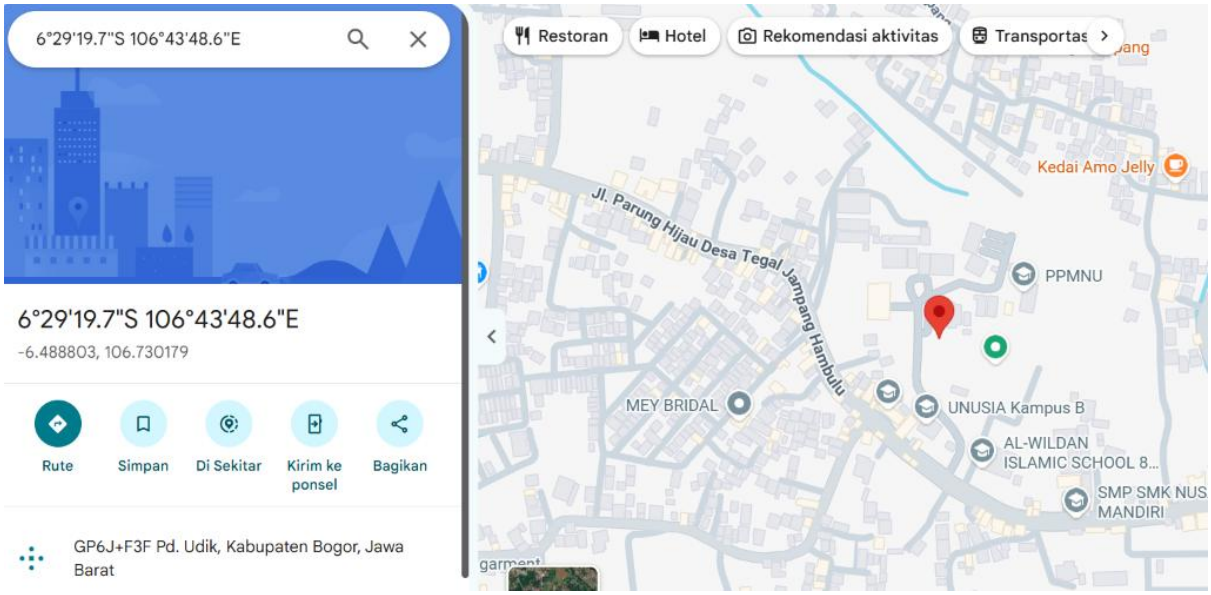
```
else:
    print("Koordinat GPS tidak lengkap atau tidak tersedia.")
```

Cabang `else` milik `if` pada baris 16. Cabang ini berjalan bila blok GPS kosong atau tidak memuat lintang dan bujur.

4. Hasil Program



Gambar 14. Hasil program tahap 6 (tabel koordinat, tautan Google Maps, dan pesan peta)



Gambar 15. Tampilan titik koordinat pada Google Maps

5. Analisis Hasil

Konversi koordinat. Fungsi `ke_desimal` menghasilkan nilai yang dirinci pada Tabel 4.

Tabel 4. Perhitungan konversi derajat-menit-detik ke derajat desimal

Komponen	Nilai DMS	Perhitungan	Arah	Hasil akhir
Lintang	6° 29′ 19,6904″	6 + 29/60 + 19,6904/3600 = 6,488803	S	−6,488803
Bujur	106° 43′ 48,6438″	106 + 43/60 + 48,6438/3600 = 106,730179	E	106,730179

Hasil ini sesuai dengan tabel pada Gambar 14, yaitu −6,488803° dan 106,730179°. Tanda negatif pada lintang muncul karena `GPSLatitudeRef` bernilai `S`. Tautan yang dibuat memuat koordinat dengan presisi penuh, yaitu `https://www.google.com/maps?q=-6.488802888888889,106.73017883333334`.

Visualisasi peta. Pada output notebook, bagian `try` gagal karena `folium` belum terpasang, sehingga cabang `except ImportError` berjalan dan mencetak pesan seperti terlihat pada Gambar 14. Peta interaktif tidak muncul pada notebook, dan titik yang ditampilkan pada peta mana pun bersumber dari koordinat yang tersimpan pada metadata foto.

Kaitan dengan lokasi UNUSIA Kampus B. Penjelasan pada notebook menyatakan bahwa ketika koordinat ditampilkan pada Google Maps, titik berada di sekitar kawasan UNUSIA Kampus B, Pondok Udik, Kabupaten Bogor, Jawa Barat. Tampilan peta tersebut tidak tersimpan pada output notebook, sehingga pernyataan ini bersandar pada pengamatan

penyusun dan perlu dilengkapi tangkapan layar pada Gambar 15. Pernyataan ini juga hanya menunjukkan bahwa koordinat metadata berada di sekitar kawasan tersebut. Isi foto (laptop, ponsel, dan meja) tidak memuat petunjuk lokasi yang jelas, sehingga lokasi pengambilan tidak dapat dipastikan dari gambar. Selain itu, akurasi koordinat GPS dapat dipengaruhi kondisi sinyal dan lingkungan saat foto diambil, sehingga titik pada peta dipahami sebagai perkiraan lokasi, bukan lokasi pasti.

2.7 Tahap 7 — Membuktikan Gambar Tersusun dari Pixel

1. Tujuan Kode

Mengubah gambar menjadi array NumPy, lalu memeriksa bentuk, jumlah dimensi, dan jumlah pixelnya untuk membuktikan bahwa citra tersusun dari pixel bernilai angka.

2. Screenshot Kode Program

```
# Mengubah citra menjadi array NumPy
image_array = np.array(image)

# Menampilkan bentuk array citra
print("Bentuk array citra:", image_array.shape)

# Menampilkan jumlah dimensi array
print("Jumlah dimensi:", image_array.ndim)

# Menghitung jumlah piksel dari tinggi × lebar citra
jumlah_pixel = image_array.shape[0] * image_array.shape[1]

print("Perhitungan jumlah piksel:", image_array.shape[0], "×", image_array.shape[1])
print("Jumlah piksel:", jumlah_pixel)
```

Gambar 16. Kode program tahap 7 (membuktikan citra tersusun dari pixel)

3. Penjelasan Setiap Baris Kode

Baris 1–2

```
# Mengubah citra menjadi array NumPy
image_array = np.array(image)
```

`np.array()` mengubah objek gambar Pillow menjadi array NumPy yang berisi nilai pixel. Hasilnya disimpan pada `image_array`, variabel penting yang dipakai sampai tahap terakhir.

Baris 4–5

```
# Menampilkan bentuk array citra
print("Bentuk array citra:", image_array.shape)
```

Atribut `shape` menampilkan bentuk array dalam urutan (tinggi, lebar, kanal). Urutan ini berbeda dari `image.size` pada Tahap 3 yang berurutan (lebar, tinggi), karena array menyimpan baris terlebih dahulu sehingga tinggi muncul lebih dulu.

Baris 7–8

```
# Menampilkan jumlah dimensi array
print("Jumlah dimensi:", image_array.ndim)
```

Atribut `ndim` menampilkan jumlah dimensi array.

Baris 10–11

```
# Menghitung jumlah piksel dari tinggi × lebar citra
jumlah_pixel = image_array.shape[0] * image_array.shape[1]
```

Indeks `[0]` mengambil tinggi (jumlah baris pixel) dan `[1]` mengambil lebar (jumlah kolom pixel). Keduanya dikalikan untuk mendapatkan jumlah pixel. Kanal tidak ikut dikalikan karena tiga kanal adalah komponen warna dari satu pixel yang sama. Variabel `jumlah_pixel` ditulis ulang dan menimpa nilai dari Tahap 3 dengan hasil yang sama.

Baris 13–14

```
print("Perhitungan jumlah piksel:", image_array.shape[0], "×",
image_array.shape[1])
print("Jumlah piksel:", jumlah_pixel)
```

Baris 13 menampilkan perhitungan (tinggi × lebar) dan baris 14 menampilkan hasilnya.

4. Hasil Program

```
Bentuk array citra: (4160, 3120, 3)
Jumlah dimensi: 3
Perhitungan jumlah piksel: 4160 × 3120
Jumlah piksel: 12979200
```

Gambar 17. Hasil program tahap 7

5. Analisis Hasil

Foto ternyata dapat dibaca sebagai array tiga dimensi berbentuk (4160, 3120, 3), yaitu 4160 baris, 3120 kolom, dan 3 nilai (R, G, B) pada setiap posisi. Jumlah pixel adalah 4160 × 3120 = 12.979.200, sama dengan hasil Tahap 3 yang dihitung dari `image.size`. Karena setiap pixel memiliki tiga nilai, array menyimpan 12.979.200 × 3 = 38.937.600 nilai angka. Hasil ini membuktikan bahwa foto digital memiliki struktur data numerik yang tersusun dari pixel dan nilai warna.

Catatan. Notebook tidak menampilkan tipe data array (`dtype`). Rentang nilai 0 sampai 255 pada pembahasan berikutnya mengikuti penjelasan notebook untuk citra RGB 8 bit. Seluruh

nilai yang tampil pada output memang berada pada rentang tersebut. Sebagai saran (bukan bagian implementasi), `image_array.dtype` dapat ditambahkan untuk memeriksanya langsung.

2.8 Tahap 8 — Melihat Nilai Pixel dalam Bentuk Angka

1. Tujuan Kode

Mengambil 4 × 4 pixel pertama dari array untuk melihat nilai RGB pixel secara langsung dalam bentuk angka.

2. Screenshot Kode Program



Gambar 18. Kode program tahap 8 (nilai pixel area 4 × 4)

3. Penjelasan Setiap Baris Kode

Baris 1–2

```
# Mengambil 4 × 4 piksel pertama dari citra
pixel_4x4 = image_array[:4, :4]
```

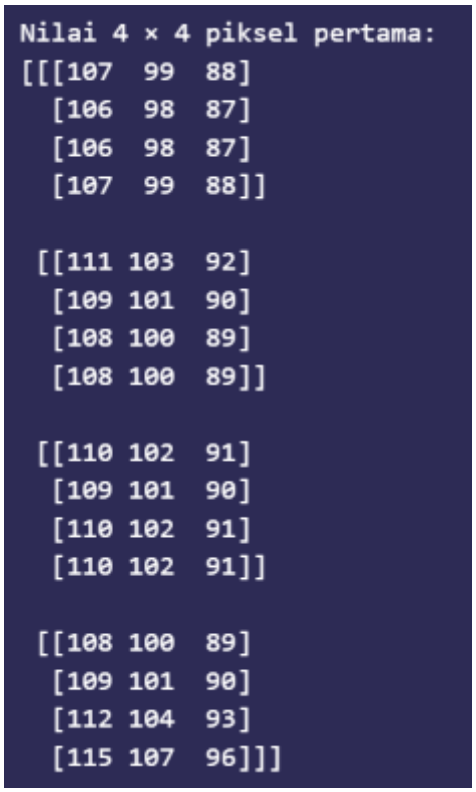
Teknik slicing pada array. `:4` pada posisi pertama memilih baris indeks 0 sampai 3, dan `:4` pada posisi kedua memilih kolom indeks 0 sampai 3 (batas akhir 4 tidak ikut diambil). Dimensi ketiga tidak disebutkan, sehingga ketiga nilai R, G, B ikut terambil. Hasilnya 4 baris × 4 kolom = 16 pixel di sudut kiri atas gambar, disimpan pada `pixel_4x4`.

Baris 4–6

```
# Menampilkan nilai piksel
print("Nilai 4 × 4 piksel pertama:")
print(pixel_4x4)
```

Baris 5 mencetak keterangan, dan baris 6 mencetak isi potongan array berupa angka.

4. Hasil Program



Gambar 19. Hasil program tahap 8 (nilai 16 pixel pertama)

5. Analisis Hasil

Output terdiri atas empat kelompok baris, dan setiap pixel ditulis sebagai tiga angka dalam urutan R, G, B. Pixel pertama, yaitu koordinat (0, 0), bernilai [107 99 88], yang berarti R = 107, G = 99, dan B = 88. Pada 16 pixel ini, nilai R berkisar 106 sampai 115, G berkisar 98 sampai 107, dan B berkisar 87 sampai 96. Nilainya saling berdekatan karena pixel bertetangga pada area yang sama biasanya bermiripan warna. Selain itu, pada setiap pixel nilai R selalu lebih besar 8 daripada G, dan G lebih besar 11 daripada B, sehingga area ini berwarna keabuan kecokelatan yang hanya berbeda tingkat kecerahannya. Hasil ini menunjukkan langsung bahwa pixel pada foto adalah angka.

2.9 Tahap 9 — Representasi RGB Setiap Pixel

1. Tujuan Kode

Memisahkan nilai setiap pixel pada area 4 × 4 menjadi komponen R, G, dan B, serta menampilkannya bersama koordinat pixel.

2. Screenshot Kode Program

```
# Mengambil 4 x 4 piksel pertama
pixel_4x4 = image_array[:4, :4]

# Menampilkan nilai RGB setiap piksel
for baris in range(4):
    for kolom in range(4):
        r, g, b = pixel_4x4[baris, kolom]
        print(f"Pixel ({baris}, {kolom}) → R={r}, G={g}, B={b}")
```

Gambar 20. Kode program tahap 9 (nilai RGB setiap pixel)

3. Penjelasan Setiap Baris Kode

Baris 1–2

```
# Mengambil 4 × 4 piksel pertama
pixel_4x4 = image_array[:4, :4]
```

Mengambil kembali 16 pixel pertama dengan slicing yang sama seperti Tahap 8.

Baris 4–5

```
# Menampilkan nilai RGB setiap piksel
for baris in range(4):
```

Perulangan luar. `range(4)` menghasilkan 0, 1, 2, 3, sehingga `baris` menjalankan empat baris pixel.

Baris 6

```
    for kolom in range(4):
```

Perulangan dalam. Untuk setiap baris, `kolom` juga bernilai 0 sampai 3. Dua perulangan bersarang menghasilkan 4 × 4 = 16 kombinasi koordinat, dari (0, 0) sampai (3, 3).

Baris 7

```
        r, g, b = pixel_4x4[baris, kolom]
```

Mengambil satu pixel berdasarkan indeks baris dan kolom. Hasilnya tiga angka yang langsung dipisahkan ke variabel `r`, `g`, dan `b` (unpacking).

Baris 8

```
        print(f"Pixel ({baris}, {kolom}) → R={r}, G={g}, B={b}")
```

Mencetak koordinat dan nilai R, G, B dengan f-string. Nilai di dalam kurung kurawal diganti dengan isi variabelnya.

4. Hasil Program

```
Pixel (0, 0) → R=107, G=99, B=88
Pixel (0, 1) → R=106, G=98, B=87
Pixel (0, 2) → R=106, G=98, B=87
Pixel (0, 3) → R=107, G=99, B=88
Pixel (1, 0) → R=111, G=103, B=92
Pixel (1, 1) → R=109, G=101, B=90
Pixel (1, 2) → R=108, G=100, B=89
Pixel (1, 3) → R=108, G=100, B=89
Pixel (2, 0) → R=110, G=102, B=91
Pixel (2, 1) → R=109, G=101, B=90
Pixel (2, 2) → R=110, G=102, B=91
Pixel (2, 3) → R=110, G=102, B=91
Pixel (3, 0) → R=108, G=100, B=89
Pixel (3, 1) → R=109, G=101, B=90
Pixel (3, 2) → R=112, G=104, B=93
Pixel (3, 3) → R=115, G=107, B=96
```

Gambar 21. Hasil program tahap 9 (nilai RGB 16 pixel)

5. Analisis Hasil

Output menampilkan 16 pixel dari (0, 0) sampai (3, 3). Nilainya sama dengan matriks pada Tahap 8; misalnya pixel (1, 0) bernilai R = 111, G = 103, B = 92, sama dengan elemen pertama baris kedua pada matriks Tahap 8. Pixel (0, 0) bernilai R = 107, G = 99, B = 88, sedangkan pixel (3, 3) bernilai R = 115, G = 107, B = 96. Setiap komponen berada pada rentang 0 sampai 255 dan menyatakan intensitas, bukan persentase. Satu pixel bukan tiga pixel yang berbeda, melainkan satu titik yang memiliki tiga nilai kanal. Ketiga nilai itu bersama-sama menentukan warna pixel tersebut.

2.10 Tahap 10 — Mengubah Sebagian Gambar Menjadi Matriks Angka

1. Tujuan Kode

Menampilkan sebagian citra berukuran 5 × 5 pixel sebagai contoh matriks angka. Hanya sebagian yang ditampilkan karena seluruh gambar terlalu besar untuk dicetak.

2. Screenshot Kode Program

```
# Mengambil sebagian citra, yaitu 5 × 5 piksel pertama
matriks_5x5 = image_array[:5, :5]

# Menampilkan matriks angka
print("Matriks RGB 5 × 5 piksel:")
print(matriks_5x5)
```

Gambar 22. Kode program tahap 10 (matriks pixel 5 × 5)

3. Penjelasan Setiap Baris Kode

Baris 1–2

```
# Mengambil sebagian citra, yaitu 5 × 5 piksel pertama
matriks_5x5 = image_array[:5, :5]
```

Mengambil baris indeks 0–4 dan kolom indeks 0–4 dari `image_array` (batas akhir tidak ikut). Hasilnya 25 pixel yang disimpan pada `matriks_5x5`. Caranya sama dengan Tahap 8, hanya areanya diperluas dari 4 × 4 menjadi 5 × 5.

Baris 4–6

```
# Menampilkan matriks angka
print("Matriks RGB 5 × 5 piksel:")
print(matriks_5x5)
```

Baris 5 mencetak keterangan, dan baris 6 mencetak matriks. Setiap elemen berupa satu pixel dengan tiga nilai RGB.

4. Hasil Program

```
Matriks RGB 5 × 5 piksel:
[[[107  99  88]
  [106  98  87]
  [106  98  87]
  [107  99  88]
  [107  99  88]]

 [[111 103  92]
  [109 101  90]
  [108 100  89]
  [108 100  89]
  [107  99  88]]

 [[110 102  91]
  [109 101  90]
  [110 102  91]
  [110 102  91]
  [108 100  89]]

 [[108 100  89]
  [109 101  90]
  [112 104  93]
  [115 107  96]
  [114 106  95]]

 [[111 103  92]
  [112 104  93]
  [116 108  97]
  [119 111 100]
  [119 111 100]]]
```

Gambar 23. Hasil program tahap 10 (matriks RGB 5 × 5)

5. Analisis Hasil

Output menampilkan 5 baris dan 5 kolom pixel, dengan setiap pixel berisi tiga angka. Pixel pertama tetap [107 99 88] dan pixel di ujung kanan bawah (koordinat (4, 4)) bernilai [119 111 100]. Empat baris dan empat kolom pertama sama dengan hasil Tahap 8, sehingga potongan 4 × 4 tersebut merupakan bagian dari matriks yang lebih besar ini. Pada potongan ini, nilai cenderung naik ke arah kanan bawah, misalnya nilai R dari 107 pada (0, 0) menjadi 119 pada (4, 4), yang menunjukkan perubahan kecerahan yang bertahap. Pola selisih yang sama dengan Tahap 8 (R lebih besar 8 daripada G dan G lebih besar 11 daripada B) juga berlaku pada seluruh 25 pixel. Dengan demikian, citra digital dapat dipandang sebagai matriks angka. Seluruh foto

tersimpan dalam `image_array`, sedangkan tahap ini hanya memperlihatkan sebagian kecilnya agar mudah dibaca.

2.11 Tahap 11 — Hubungan Koordinat Pixel, Koordinat, dan Warna

1. Tujuan Kode

Mengambil nilai RGB pada beberapa koordinat pixel yang berbeda, menyusunnya dalam tabel, dan menampilkan warna setiap pixel agar hubungan antara koordinat, nilai numerik, dan warna dapat diamati.

2. Screenshot Kode Program

Kode ditampilkan dalam dua bagian dengan nomor baris yang berlanjut.

```
# Menentukan beberapa koordinat pixel yang akan dianalisis
koordinat_pixel = [
    (0, 0),
    (0, 100),
    (100, 100),
    (200, 200),
    (500, 500)
]

# Menyimpan hasil analisis pixel
data_pixel = []

for baris, kolom in koordinat_pixel:
    # Mengambil nilai RGB berdasarkan koordinat
    r, g, b = image_array[baris, kolom]

    # Menyimpan koordinat dan nilai RGB
    data_pixel.append({
        "Koordinat (Baris, Kolom)": f"({baris}, {kolom})",
        "Red (R)": int(r),
        "Green (G)": int(g),
        "Blue (B)": int(b),
        "Warna RGB": f"({r}, {g}, {b})"
    })
```

Gambar 24. Kode program tahap 11, bagian 1 (pengambilan nilai RGB dan tabel)

```
# Menampilkan hasil dalam bentuk tabel

tabel_pixel = pd.DataFrame(data_pixel)
display(tabel_pixel)

# Menampilkan warna dari setiap koordinat pixel
plt.figure(figsize=(8, 2))

for i, (baris, kolom) in enumerate(koordinat_pixel):
    r, g, b = image_array[baris, kolom]

    plt.subplot(1, len(koordinat_pixel), i + 1)
    plt.imshow([[[r, g, b]]])
    plt.title(f"({baris}, {kolom})")
    plt.axis("off")

plt.suptitle("Warna Pixel Berdasarkan Koordinat")
plt.tight_layout()
plt.show()
```

Gambar 25. Kode program tahap 11, bagian 2 (visualisasi warna pixel)

3. Penjelasan Setiap Baris Kode

Baris 1–8

```
# Menentukan beberapa koordinat pixel yang akan dianalisis
koordinat_pixel = [
    (0, 0),
    (0, 100),
    (100, 100),
    (200, 200),
    (500, 500)
]
```

Membuat daftar `koordinat_pixel` berisi lima pasangan koordinat dalam bentuk (baris, kolom). Urutan ini berarti angka pertama adalah posisi vertikal dan angka kedua posisi horizontal.

Baris 10–11

```
# Menyimpan hasil analisis pixel
data_pixel = []
```

Membuat daftar kosong untuk menampung hasil setiap pixel.

Baris 13–15

```
for baris, kolom in koordinat_pixel:
    # Mengambil nilai RGB berdasarkan koordinat
    r, g, b = image_array[baris, kolom]
```

Perulangan membaca satu koordinat pada setiap putaran dan langsung memisahkannya ke `baris` dan `kolom`. Baris 15 mengambil pixel pada koordinat tersebut dan memisahkan nilainya ke `r`, `g`, dan `b`.

Baris 17–24

```
    # Menyimpan koordinat dan nilai RGB
    data_pixel.append({
        "Koordinat (Baris, Kolom)": f"({baris}, {kolom})",
        "Red (R)": int(r),
        "Green (G)": int(g),
        "Blue (B)": int(b),
        "Warna RGB": f"({r}, {g}, {b})"
    })
```

Menambahkan satu kamus ke `data_pixel` pada setiap putaran. Kunci kamus nantinya menjadi nama kolom tabel. `int(r)`, `int(g)`, dan `int(b)` mengubah nilai NumPy menjadi bilangan bulat Python biasa, sedangkan kolom `"Warna RGB"` menyimpan ketiga nilai sebagai teks dalam bentuk (R, G, B).

Baris 26–27

```
# Menampilkan hasil dalam bentuk tabel
import pandas as pd
```

Mengimpor Pandas kembali. Library ini sudah diimpor pada Tahap 1, sehingga baris ini tidak diperlukan tetapi tidak menimbulkan kesalahan.

Baris 29–30

```
tabel_pixel = pd.DataFrame(data_pixel)
display(tabel_pixel)
```

`pd.DataFrame(data_pixel)` mengubah daftar kamus menjadi tabel, dan `display()` menampilkannya.

Baris 32–33

```
# Menampilkan warna dari setiap koordinat pixel
plt.figure(figsize=(8, 2))
```

Membuat kanvas berukuran 8 × 2 inci untuk menampilkan lima warna secara berdampingan.

Baris 35–36

```
for i, (baris, kolom) in enumerate(koordinat_pixel):
    r, g, b = image_array[baris, kolom]
```

`enumerate()` memberikan nomor urut `i` (dimulai dari 0) bersama pasangan koordinatnya. Baris 36 mengambil kembali nilai RGB pixel pada koordinat tersebut.

Baris 38–41

```
    plt.subplot(1, len(koordinat_pixel), i + 1)
    plt.imshow([[[r, g, b]]])
    plt.title(f"({baris}, {kolom})")
    plt.axis("off")
```

`plt.subplot(1, 5, i + 1)` membagi kanvas menjadi 1 baris dan 5 kolom lalu memilih panel ke-`i + 1` (penomoran panel dimulai dari 1). `plt.imshow([[[r, g, b]]])` menampilkan daftar bersarang berukuran 1 × 1 × 3, yaitu gambar yang hanya berisi satu pixel dan terlihat sebagai satu bidang warna. `plt.title()` menampilkan koordinat di atas bidang tersebut, dan `plt.axis("off")` menyembunyikan sumbu.

Baris 43–45

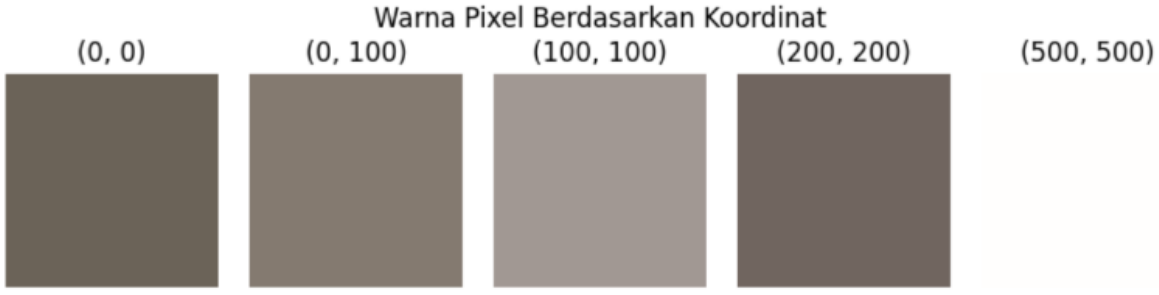
```
plt.suptitle("Warna Pixel Berdasarkan Koordinat")
plt.tight_layout()
plt.show()
```

`plt.suptitle()` memberi judul untuk seluruh kanvas, `plt.tight_layout()` merapikan jarak antarpanel, dan `plt.show()` menampilkan hasilnya.

4. Hasil Program

	Koordinat (Baris, Kolom)	Red (R)	Green (G)	Blue (B)	Warna RGB
0	(0, 0)	107	99	88	(107, 99, 88)
1	(0, 100)	132	122	112	(132, 122, 112)
2	(100, 100)	161	152	147	(161, 152, 147)
3	(200, 200)	112	101	95	(112, 101, 95)
4	(500, 500)	255	254	253	(255, 254, 253)

Gambar 26. Tabel nilai RGB pada lima koordinat pixel (tahap 11)



Gambar 27. Warna pixel berdasarkan koordinat (tahap 11)

5. Analisis Hasil

Tabel pada Gambar 26 memuat lima koordinat, dan setiap koordinat memiliki nilai RGB yang berbeda, seperti dirangkum pada Tabel 5. Warna pada Gambar 27 adalah warna yang dibentuk oleh nilai RGB tersebut.

Tabel 5. Nilai RGB pada koordinat pixel yang dianalisis

Koordinat (baris, kolom)	R	G	B	Pengamatan pada warna
(0, 0)	107	99	88	Warna keabuan kecokelatan yang gelap; sama dengan pixel (0, 0) pada Tahap 8 sampai 10.
(0, 100)	132	122	112	Lebih terang daripada (0, 0), dengan nuansa kecokelatan yang serupa.
(100, 100)	161	152	147	Abu-abu terang dengan sedikit nuansa hangat.
(200, 200)	112	101	95	Keabuan kecokelatan yang gelap.
(500, 500)	255	254	253	Hampir putih; R mendekati batas maksimum 255.

Hasil ini menunjukkan hubungan tiga hal: koordinat menentukan pixel yang diambil, nilai RGB pada koordinat itu berupa tiga angka, dan ketiga angka tersebut menentukan warna yang tampil. Nilai ketiga kanal yang makin besar menghasilkan warna yang makin terang, dan nilai ketiga kanal yang hampir sama (seperti 255, 254, 253) menghasilkan warna netral mendekati putih. Penyebutan warna pada tabel bersifat deskriptif berdasarkan tampilan, bukan hasil perhitungan.

2.12 Tahap 12 — Menandai Letak Pixel pada Foto Asli

1. Tujuan Kode

Menampilkan foto asli dan menandai posisi kelima pixel pada Tahap 11 agar terlihat letaknya pada foto.

2. Screenshot Kode Program

```
# Menampilkan foto asli dan menandai posisi pixel yang dianalisis
plt.figure(figsize=(8, 6))
plt.imshow(image)

for baris, kolom in koordinat_pixel:
    plt.scatter(kolom, baris, color="red", s=40)
    plt.text(kolom + 20, baris + 20,
             f"({baris}, {kolom})",
             color="red", fontsize=8)

plt.title("Letak Pixel yang Dianalisis pada Foto Asli")
plt.axis("on")
plt.show()
```

Gambar 28. Kode program tahap 12 (menandai posisi pixel pada foto)

3. Penjelasan Setiap Baris Kode

Baris 1–3

```
# Menampilkan foto asli dan menandai posisi pixel yang dianalisis
plt.figure(figsize=(8, 6))
plt.imshow(image)
```

Membuat kanvas 8 × 6 inci dan menampilkan foto asli (`image`) sebagai latar.

Baris 5–6

```
for baris, kolom in koordinat_pixel:
    plt.scatter(kolom, baris, color="red", s=40)
```

Perulangan memakai kembali daftar `koordinat_pixel` dari Tahap 11. `plt.scatter()` menggambar titik merah. Perhatikan urutan argumennya, yaitu `kolom` lebih dulu lalu `baris`, karena sumbu horizontal (x) pada grafik adalah kolom pixel dan sumbu vertikal (y) adalah baris pixel. Parameter `s=40` mengatur ukuran titik.

Baris 7–9

```
    plt.text(kolom + 20, baris + 20,
             f"({baris}, {kolom})",
             color="red", fontsize=8)
```

`plt.text()` menulis label koordinat berwarna merah dengan ukuran huruf 8, digeser 20 satuan pixel ke kanan dan ke bawah dari titik agar tidak menimpa titik.

Baris 11–13

```
plt.title("Letak Pixel yang Dianalisis pada Foto Asli")
plt.axis("on")
plt.show()
```

Memberi judul, menampilkan sumbu (`"on"`, berbeda dari tahap sebelumnya yang menyembunyikannya) agar skala pixel terlihat, lalu menampilkan hasilnya.

4. Hasil Program



Gambar 29. Letak pixel yang dianalisis pada foto asli (tahap 12)

5. Analisis Hasil

Sumbu pada gambar memperlihatkan skala pixel foto asli, sekitar 0 sampai 3000 pada sumbu horizontal dan 0 sampai 4000 pada sumbu vertikal. Kelima titik berada di sudut kiri atas foto, pada rentang baris 0–500 dan kolom 0–500. Nilai 500 tersebut hanya sekitar 12% dari tinggi (500/4160) dan 16% dari lebar (500/3120) foto. Artinya, kelima warna pada Tahap 11 hanya mewakili sebagian kecil foto dan tidak menggambarkan seluruh warna foto. Titik (500, 500) berada pada bagian foto yang berwarna terang, sesuai dengan nilai RGB-nya yang mendekati putih. Label (0, 0) dan (0, 100) terletak berdekatan sehingga sebagian tampak saling tumpang tindih.

2.13 Tahap 13 — Rekonstruksi Gambar dan Pemeriksaan Kesamaan

1. Tujuan Kode

Membentuk kembali gambar dari array angka lengkap (`image_array`) dan membandingkannya dengan gambar asli, baik secara visual maupun dengan membandingkan seluruh nilai pixel menggunakan `np.array_equal()`.

2. Screenshot Kode Program

Kode ditampilkan dalam dua bagian dengan nomor baris yang berlanjut.

```
# Membentuk kembali gambar dari array angka
gambar_rekonstruksi = Image.fromarray(image_array)

# Membandingkan data pixel gambar asli dan hasil rekonstruksi
data_asli = np.array(image)
data_rekonstruksi = np.array(gambar_rekonstruksi)
identik = np.array_equal(data_asli, data_rekonstruksi)

# Menampilkan kedua gambar berdampingan
plt.figure(figsize=(10, 6))

plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Gambar Asli")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gambar_rekonstruksi)
plt.title("Gambar Hasil Rekonstruksi")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Gambar 30. Kode program tahap 13, bagian 1 (rekonstruksi dan tampilan berdampingan)

```
# Menampilkan hasil perbandingan
print("Apakah kedua gambar identik?", identik)

if identik:
    print("Hasil: Kedua gambar identik.")
else:
    print("Hasil: Kedua gambar tidak identik.")
```

Gambar 31. Kode program tahap 13, bagian 2 (hasil perbandingan)

3. Penjelasan Setiap Baris Kode

Baris 1–2

```
# Membentuk kembali gambar dari array angka
gambar_rekonstruksi = Image.fromarray(image_array)
```

`Image.fromarray()` adalah fungsi Pillow yang berkebalikan dengan `np.array()`, yaitu membentuk objek gambar dari array. Array yang dipakai adalah `image_array` secara lengkap (4160 × 3120 pixel), bukan potongan contoh pada tahap sebelumnya. Karena array berbentuk (tinggi, lebar, 3), Pillow menafsirkannya sebagai gambar RGB (untuk data 8 bit per kanal). Hasilnya disimpan pada `gambar_rekonstruksi`.

Baris 4–7

```
# Membandingkan data pixel gambar asli dan hasil rekonstruksi
data_asli = np.array(image)
data_rekonstruksi = np.array(gambar_rekonstruksi)
identik = np.array_equal(data_asli, data_rekonstruksi)
```

Baris 5 dan 6 mengubah gambar asli dan gambar hasil rekonstruksi menjadi array. `np.array_equal()` pada baris 7 memeriksa apakah kedua array memiliki bentuk yang sama dan seluruh elemennya sama persis. Hasilnya `True` atau `False`, disimpan pada `identik`. Perbandingan ini mencakup seluruh nilai pixel, bukan hanya sampel.

Baris 9–10

```
# Menampilkan kedua gambar berdampingan
plt.figure(figsize=(10, 6))
```

Membuat kanvas 10 × 6 inci agar dua gambar dapat ditampilkan berdampingan.

Baris 12–15

```
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Gambar Asli")
plt.axis("off")
```

`plt.subplot(1, 2, 1)` membagi kanvas menjadi 1 baris dan 2 kolom lalu memilih panel pertama (kiri). Panel ini diisi gambar asli dengan judul `"Gambar Asli"` dan tanpa sumbu.

Baris 17–20

```
plt.subplot(1, 2, 2)
plt.imshow(gambar_rekonstruksi)
plt.title("Gambar Hasil Rekonstruksi")
plt.axis("off")
```

Memilih panel kedua (kanan) dan mengisinya dengan gambar hasil rekonstruksi berjudul `"Gambar Hasil Rekonstruksi"`, juga tanpa sumbu.

Baris 22–23

```
plt.tight_layout()
plt.show()
```

`plt.tight_layout()` merapikan jarak antarpanel agar judul dan gambar tidak saling menimpa, dan `plt.show()` menampilkan kedua gambar.

Baris 25–26

```
# Menampilkan hasil perbandingan
print("Apakah kedua gambar identik?", identik)
```

Mencetak hasil pemeriksaan kesamaan yang tersimpan pada `identik`.

Baris 28–31

```
if identik:
    print("Hasil: Kedua gambar identik.")
else:
    print("Hasil: Kedua gambar tidak identik.")
```

Struktur kondisi yang mencetak kesimpulan dalam kalimat. Jika `identik` bernilai `True`, yang dicetak adalah kalimat pada baris 29, dan jika `False`, yang dicetak adalah kalimat pada baris 31.

4. Hasil Program



Gambar 32. Gambar asli dan gambar hasil rekonstruksi berdampingan (tahap 13)

```
Apakah kedua gambar identik? True
Hasil: Kedua gambar identik.
```

Gambar 33. Hasil pemeriksaan kesamaan pada tahap 13

5. Analisis Hasil

Kedua gambar tampak sama pada Gambar 32. Pemeriksaan `np.array_equal()` menghasilkan `True` dan program mencetak bahwa kedua gambar identik (Gambar 33). Artinya,

bentuk kedua array sama dan seluruh nilai pixelnya (12.979.200 pixel × 3 kanal) sama persis, sehingga proses rekonstruksi tidak mengubah nilai pixel. Hasil ini membuktikan bahwa gambar dapat dibentuk kembali dari array angka dengan nilai pixel yang sama seperti gambar aslinya.

Batasan interpretasi. Pemeriksaan ini dilakukan pada data di memori. Gambar hasil rekonstruksi tidak disimpan ke file pada notebook, sehingga hasil `True` tidak menyatakan apa pun tentang hasil penyimpanan ulang ke format lain. Notebook juga tidak menguji apakah metadata EXIF ikut terbawa pada gambar hasil rekonstruksi.

BAB III KESIMPULAN

Berdasarkan seluruh tahap analisis pada notebook, tujuan projek dapat dijawab sebagai berikut.

1. **Informasi dasar.** Foto yang dianalisis bernama `foto.jpg`, berformat JPEG, berukuran file 3.452.320 byte (sekitar 3,45 MB), beresolusi 3120 × 4160 pixel, bermode warna RGB, dengan jumlah pixel 12.979.200.
2. **Metadata EXIF.** Terdapat 14 entri EXIF utama, antara lain `Make` vivo, `Model` V2204, `Software` MediaTek Camera Application, dan `DateTime` 2026:10:06 16:18:26. Angka pada `GPSInfo` (1092) dan `ExifOffset` (346) adalah penunjuk ke blok metadata lain, bukan data isi. Nilai `Orientation` 0 berada di luar nilai standar dan tidak ditafsirkan.
3. **Metadata GPS.** Blok GPS berhasil dibaca dan memuat lintang 6° 29′ 19,6904″ S, bujur 106° 43′ 48,6438″ E, dan ketinggian 155,0 meter. Konversi menghasilkan koordinat desimal −6,488803; 106,730179 yang dipakai untuk membuat tautan Google Maps. Peta interaktif folium tidak tampil karena library belum terpasang. Menurut penjelasan notebook, titik koordinat berada di sekitar kawasan UNUSIA Kampus B, Pondok Udik, Kabupaten Bogor, tetapi koordinat metadata hanya menunjukkan perkiraan lokasi dan tidak dipakai untuk menyatakan lokasi pasti pengambilan foto.
4. **Foto sebagai array.** Foto dapat dibaca sebagai array tiga dimensi berbentuk (4160, 3120, 3) dengan 12.979.200 pixel dan tiga kanal warna R, G, dan B.
5. **Pixel, koordinat, dan warna.** Setiap pixel memiliki posisi (baris, kolom) dan tiga nilai RGB. Contohnya pixel (0, 0) bernilai R = 107, G = 99, B = 88, sedangkan pixel (500, 500) bernilai R = 255, G = 254, B = 253 dan tampil hampir putih.
6. **Rekonstruksi.** Gambar yang dibentuk kembali dengan `Image.fromarray()` menghasilkan `np.array_equal()` bernilai `True`, sehingga seluruh nilai pixelnya identik dengan gambar asli.

Dengan demikian, projek ini membuktikan bahwa foto digital tersusun dari pixel bernilai angka yang dapat dibaca, diamati, dan dipakai untuk membentuk kembali gambar yang identik, serta bahwa file foto juga menyimpan metadata, termasuk koordinat lokasi, yang perlu dipahami dan ditafsirkan dengan hati-hati.

Keterbatasan. Analisis hanya mencakup satu foto. Notebook tidak menampilkan tipe data array, tidak membaca blok EXIF lanjutan (sub-IFD), dan tidak menyimpan tampilan peta pada outputnya.